A

Project Report titled

"Automatic Parking System"

Submitted to Department of

Data science and artificial intelligence



Hindi Vidya Prachar Samiti's RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE & COMMERCE

For Partial Fulfillment for Degree of Bachelor

of Science (DSAI) 2025 – 2026 In the Subject

Project (V Semester) Submitted by

Kunal Gupta - 10248

Mohd Suhail Khan - 10253

indi Vidya Prachar Samiti's RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE & COMMERCE

CERTIFICATE

This is to certify that Kunal gupta(10248) and Mohd Suhail Khan(10253), a third year student of Bachelor of Data Science and Artifical Intelligence has successfully completed the project entitled "Automatic Parking System" as a part of academic in the subject Project (V Semester) which is approved for degree of Bachelor of Data Science and Artifical Intelligence

Project Guide

Examiner

Coordinator

Hindi Vidya Prachar Samiti's RAMNIRANJAN JHUNJHUNWALA COLLEGE OF ARTS, SCIENCE & COMMERCE

PLAGIARISM CERTIFICATE

This is to certify that MRs,KUNAL GUPTA and MRs.MOHD SUHAIL KHAN a final year student of Bachelor of Data science and Artifical Intelligence from University of Mumbai has submitted the project book content for plagiarism check. It was found that his content is good, as per format and the duplication is under UGC Recommended rules given in UGC Public Notice on Draft UGC Regulations-2017(No.F.1-8/2010 (CPP-II)).

In-Charge
Plagiarism Committee

Project Guide

Coordinator

Date:

Stamp:

PREFACE

It gives me great pleasure to present this project documentation and application developed as part of my final year project. The idea originated from the growing challenges of parking management in urban areas, such as traffic congestion, time wastage, and lack of automation.

Existing parking systems are often costly, relying on licensed software and expensive infrastructure, which makes them unsuitable for small parking owners. To address this, I designed an open-source automatic parking slot detection and online booking platform using YOLO for slot and number plate detection, Streamlit for prototype development, and RDBMS for database management.

The project aims to provide a cost-effective, scalable, and automated solution that reduces human effort while improving efficiency in parking management.

This black book reflects the journey of developing the system and the effort put into making it practical and user-friendly.

ACKNOWLEDGEMENT

take this opportunity to express my deep sense of gratitude to all those who have contributed to the successful completion of this project and black book documentation.

First and foremost, I would like to sincerely thank my project guide for their constant support, valuable suggestions, and expert guidance throughout the project. Their encouragement, timely feedback, and technical insights have been instrumental in shaping the project and enhancing my understanding of the subject.

I am also grateful to the Head of Department and all faculty members of my institute for providing me with the opportunity to carry out this project, as well as for their encouragement, cooperation, and availability whenever I needed help. Their teaching and guidance during the course of my studies have played an important role in the successful comple

I extend my heartfelt thanks to my classmates and friends who pro feedback, shared ideas, and supported me during various stages collaboration and motivation kept me focused and inspired.

I also wish to acknowledge the immense moral support and encouragement given by my family. Their patience, motivation, and belief in me have always been a source of strength and inspiration. Without their support, the successful completion of this project would not have been possible.

Finally, I thank everyone who directly or indirectly contributed to this project. This work is a reflection of the collective guidance, support, and encouragement I received, and I remain truly grateful to all.

INDEX

CHP No.	CONTENT	Page No.
1.	INTRODUCTION	
	1.1 BACKGROUND	1
	1.2 OBJECTIVES	1
	1.3 PURPOSE	1
	1.4 SCOPE	1
	1.5 APPLICABILITY	1
2.	SURVEY OF TECHNOLOGIES	
	2.1 AVAILABLE TECHNOLOGIES	2
	2.2 TECHNOLOGIES USED	3
3.	REQUIREMENT ANALYSIS	
	3.1 PROBLEM DEFINATION	4
	3.2 REQUIREMENT SPECIFICATION	4
	3.3 PLANNING AND SCHEDULING	5
	3.4 SOFTWARE AND HARDWARE REQUIREMENTS	6
4.	SYSTEM DESIGN	
	4.1 MODULE	7
	4.2 FEATURE	8
	4.3 DATA SCHEMA	11
	4.4 EVENT TABLE	15
		17

5.	***IMPLEMENTATION AND TESTING***	
	5.1 IMPLEMENTATION APPROACHES	30
	5.2 TESTING APPROACHES	33
6	***RESULTS AND DISCUSSION***	
	6.1 TEST REPORTS	42
	6.2 USER DOCUMENTATION	44
7	***CONCLUSION***	
	7.1 CONCLUSION	49
	7.2 LIMITATIONS OF THE SYSTEM	49
	7.3 FUTURE SCOPE	50

CHAPTER 1
INTRODUCTION

Parking has become one of the most pressing issues in modern cities. With the increase in urbanization, population growth, and the rising number of vehicles on roads, the problem of finding an available parking slot has intensified. In most cases, drivers waste a significant amount of time and fuel searching for free parking slots. This not only leads to personal frustration but also contributes to traffic congestion, increased pollution, and inefficient resource usage.

Conventional parking management systems depend heavily on manual monitoring, ticketing, and supervision, which are prone to human errors and inefficiencies. Moreover, many of the existing automated systems are based on licensed software that comes with very high setup and maintenance costs. Smaller parking areas such as those in local markets, offices, and community spaces cannot afford such solutions.

To address this problem, we have developed an open-source parking slot detection and online booking platform. The system uses computer vision models integrated with CCTV cameras to monitor parking lots, detect available slots, and recognize vehicle number plates. In parallel, a user-friendly web-based booking platform allows drivers to check availability and reserve their slot before reaching the parking lot.

The uniqueness of our solution lies in its cost-effectiveness and open-source adaptability, which makes it accessible for both large and small parking operators. By eliminating the dependency on licensed, expensive software, the system democratizes access to smart parking management. The solution is scalable, reliable, and contributes to reducing human effort, operational costs, and environmental impact.

CHAPTER 1
INTRODUCTION

Objectives

The main aim of this project is to build a practical, affordable, and automated parking slot management system. To achieve this, the specific objectives are:

- To develop an automated mechanism that detects parking slot availability using CCTV footage and computer vision techniques.
- To enable real-time updates of slot availability on a centralized platform accessible by end-users.
- To provide an online booking system so that users can reserve slots before arriving at the parking location.
- To design the system with open-source tools and frameworks to avoid expensive licensing costs.
- To minimize human involvement in daily operations, thereby reducing human errors and saving manpower costs.
- To provide scalability so that the same system can be implemented in small parking spaces as well as large-scale public parking areas.
- To encourage smoother traffic management by reducing unnecessary waiting time and congestion near parking zones.

Scope

The scope of this project extends beyond just building a prototype. It envisions the future of smart cities, where parking systems are seamlessly integrated with urban mobility.

- For drivers, the system provides convenience by allowing them to locate, check, and reserve parking slots from their smartphone or web browser.
- For parking owners, it reduces the dependency on manual staff and makes slot monitoring more efficient.
- For society, it reduces unnecessary vehicle movement, thereby lowering traffic congestion and fuel wastage.
- For future scalability, the system can be integrated with IoT devices, payment gateways, and smart navigation apps like Google Maps.

CHAPTER 1

INTRODUCTION

Application

The applicability of this system is vast because parking is a universal issue. Some major domains of use include:

- Commercial Spaces: Shopping malls, multiplexes, and office complexes can use it to streamline customer parking.
- Public Transport Hubs: Metro stations, bus depots, and railway stations often suffer from high parking demand, where this system can play a critical role.
- Healthcare Institutions: Hospitals and clinics require efficient parking solutions to reduce stress for patients and visitors.
- Government & Public Utilities: Smart cities and municipal corporations can use it for community parking spaces.
- Event Venues: Stadiums, convention centers, and exhibitions can use the system to manage temporary parking surges.

CHAPTER 2

SURVEY OF TECHNOLOGIES

Available Technologies

A survey of existing parking solutions revealed that while advanced parking systems are available, they come with significant drawbacks:

1. High Licensing Costs: Most proprietary software solutions require costly licenses, making them unsuitable for small parking operators.
2. Complex Hardware: Advanced sensors and high-end devices increase installation and maintenance costs.
3. Limited Flexibility: Proprietary systems are less customizable and lock customers into vendor-specific hardware/software.

In contrast, our project leverages open-source technologies such as YOLO (computer vision), Streamlit (rapid prototyping), and Flask (deployment), combined with affordable hardware like CCTV cameras. This allows us to provide similar functionality at a fraction of the cost, with more flexibility.

Technologies Used

- YOLO (You Only Look Once): Used for real-time object detection, specifically to recognize empty and occupied parking slots, as well as number plate recognition.
- OpenCV: For processing video input from CCTV cameras and integrating detection models.
- Streamlit: Used to create the prototype user interface for testing booking and slot availability features.
- Flask: For building the backend APIs and enabling platform deployment.
- RDBMS (MySQL/PostgreSQL): To store user records, booking transactions, slot availability, and historical data.

These technologies collectively provide a robust and affordable smart parking solution.

:

CHAPTER 3

REQUIREMENT ANALYSIS

problem Definition:

Drivers waste time and fuel searching for slots, often finding that the lot is already full. This causes frustration, congestion, and inefficiency.

Requirement Specification:

- Hardware Requirements:
  - CCTV camera with a wide view of the parking lot.
  - Computer system capable of running YOLO models (mid-range CPU/GPU is sufficient).
  - Network connectivity for data transfer and user platform access.
- Software Requirements:
  - Python 3.12
  - Flask 3.1.2
  - Streamlit for prototype interface
  - YOLO model for slot detection and number plate recognition
  - RDBMS for database operations
  - OpenCV for image/video processing

Planning and Scheduling

The project was executed in structured phases:

1. Literature survey of existing parking solutions.
2. Identifying shortcomings of current licensed systems.
3. Designing the system architecture for an open-source model.
4. Building a Streamlit prototype for slot booking and visualization.
5. Training and integrating YOLO for real-time slot detection.
6. Database design for storing user, slot, and transaction records.
7. Backend integration using Flask for deployment.
8. Testing with simulated CCTV inputs and refining the model.
9. Final deployment and demonstration.

CHAPTER 3

REQUIREMENT ANALYSIS

problem Definition:

Drivers waste time and fuel searching for slots, often finding that the lot is already full. This causes frustration, congestion, and inefficiency.

Requirement Specification:

- Hardware Requirements:
  - CCTV camera with a wide view of the parking lot.
  - Computer system capable of running YOLO models (mid-range CPU/GPU is sufficient).
  - Network connectivity for data transfer and user platform access.
- Software Requirements:
  - Python 3.12
  - Flask 3.1.2
  - Streamlit for prototype interface
  - YOLO model for slot detection and number plate recognition
  - RDBMS for database operations
  - OpenCV for image/video processing

Planning and Scheduling

The project was executed in structured phases:

1. Literature survey of existing parking solutions.
2. Identifying shortcomings of current licensed systems.
3. Designing the system architecture for an open-source model.
4. Building a Streamlit prototype for slot booking and visualization.
5. Training and integrating YOLO for real-time slot detection.
6. Database design for storing user, slot, and transaction records.
7. Backend integration using Flask for deployment.
8. Testing with simulated CCTV inputs and refining the model.
9. Final deployment and demonstration.

CHAPTER 4

SYSTEM DESIGN

4.1 Basic Modules

- Presentation Layer (Front-end)
- Streamlit prototype/web interface for users to check and book parking slots.
- Allows login/registration, slot booking, and payment.
- Application Layer (Back-end)
- Flask server handles requests between front-end and database.
- YOLO model + OpenCV for slot detection and number plate recognition.
- Logic for slot availability and booking management.
- Database Layer (Data Storage)
- RDBMS (MySQL/PostgreSQL) stores user details, slot information, booking records, and payments.
- Ensures data integrity and prevents double-booking.

4.2 feature

T1. User Registration & Login
- Secure sign-up and login functionality for users.
- Each user can manage their vehicle details and booking history.

2. Real-time Slot Detection
- Integration with CCTV + YOLO model to detect whether slots are occupied or free.
- Updates slot status in real-time for users.

3. Parking Slot Availability Dashboard
- Users can view all nearby parking areas.
- Displays available slots, location, and per-hour charges.

4. Online Slot Booking
- Users can pre-book a slot before reaching the parking area.
- Reduces time wasted in searching for slots.

5. Number Plate Recognition
- YOLO + OpenCV detects and verifies vehicle number plates.
- Ensures only valid/registered vehicles can book and enter.

6. Online Payment System
- Integration of multiple payment methods (UPI, cards, cash).
- Generates digital receipts for each booking.

7. Booking History & Reports
- Users can track their past bookings and payments.
- Admin can generate reports of parking usage, revenue, and occupancy.

8. Admin Panel

CHAPTER 4

SYSTEM DESIGN

4.3 data schema

Booking	booking_id (PK)	INT	Unique booking ID
	user_id (FK)	INT	Reference to User
	slot_id (FK)	INT	Reference to Slot
	start_time	DATETIME	Booking start
	end_time	DATETIME	Booking end
	status	VARCHAR(20)	Active/Completed
Payment	payment_id (PK)	INT	Unique payment ID
	booking_id (FK)	INT	Refers to Booking
	amount	DECIMAL(7,2)	Payment amount
	payment_mode	VARCHAR(20)	UPI, Card, Cash
	status	VARCHAR(20)	Success/Failed
Admin	admin_id (PK)	INT	Admin unique ID
	name	VARCHAR(50)	Admin name
	email	VARCHAR(100)	Admin email
	password	VARCHAR(50)	Admin password

CHAPTER 4
SYSTEM DESIGN

Booking	booking_id (PK)	INT	Unique booking ID
	user_id (FK)	INT	Reference to User
	slot_id (FK)	INT	Reference to Slot
	start_time	DATETIME	Booking start
	end_time	DATETIME	Booking end
	status	VARCHAR(20)	Active/Completed
Payment	payment_id (PK)	INT	Unique payment ID
	booking_id (FK)	INT	Refers to Booking
	amount	DECIMAL(7,2)	Payment amount
	payment_mode	VARCHAR(20)	UPI, Card, Cash
	status	VARCHAR(20)	Success/Failed
Admin	admin_id (PK)	INT	Admin unique ID
	name	VARCHAR(50)	Admin name
	email	VARCHAR(100)	Admin email
	password	VARCHAR(50)	Admin password

CHAPTER 4

SYSTEM DESIGN

4.4 Event table

Event ID	Event Name	Actor	Description
E1	User Registration	User	New user creates account
E2	Login	User/Admin	System validates credentials
E3	Slot Detection Update	CCTV + YOLO	Updates DB with occupied/free status
E4	Search Parking Slot	User	User checks available slots
E5	Book Slot	User	User books available slot
E6	Payment	User	User pays for booking
E7	Booking Confirmation	System	Confirms booking in DB
E8	Cancel Booking	User/Admin	Cancels a booking
E9	Generate Report	Admin	Admin views booking/payment records

4.3 DATA DICTIONARY

1. Table Name: Users

Table Description:

This table consists of data required for authentication of the users on the website. The authentication takes place using the username and the hashed password stored in this table.

Fields	Description	Data Type	Constraint
uid	It is the unique id assigned for every authenticated user.	INT	PRIMARY KEY, NOT NULL
username	It is a unique string for every user.	VARCHAR(20)	NOT NULL
password	It is the password used to authenticate a user.	VARCHAR(20)	NOT NULL
email	It is the email address of a user.	VARCHAR(20)	NOT NULL

2. Table Name: Details

Table Description:

This table consists of data that a user can edit and add to their profile which will be visible on the profile.

Fields	Description	Data Type	Constraint
did	This is the details id unique to every user.	INT	PRIMARY KEY, NOT NULL
uid	This is the user id.	INT	FOREIGN KEY, NOT NULL
dob	This is the date of birth of the user.	DATE	NOT NULL
phone	This is the contact number of the user.	VARCHAR(10)	NOT NULL
split	This is the splitting percentage of the funds in a team	INT	NOT NULL

3. Table Name: Sessions

Table Description:

The tables contains the session related data unique to every user to maintain sessions.

Fields	Description	Data Type	Constraint
sid	This is the session id unique to a particular session of a user.	INT	PRIMARY KEY, NOT NULL
session	This is the data of the session in the json format.	JSON	NOT NULL
expires	This is the timestamp of the expiration of a session.	TIMESTAMP	

4. Table Name: Posts Table Description: The tables contains the posts of the update wall and the posts posted by the users will be

fetched using this table.

Fields	Description	Data Type	Constraint
pid	This is the unique id of every post on the update wall.	INT	PRIMARY KEY, NOT NULL
post	This is the content of the post.	TEXT	NOT NULL
uid	This is the user id of the user who has posted the post.	INT	FOREIGN KEY, NOT NULL

5. Table Name: Payment

Table Description:

The tables contain data which has data of money related transactions.

Fields	Description	Data Type	Constraints
pid	This is the payment id which will be unique for every transaction that happens.	INT	PRIMARY KEY, NOT NULL
amount	This is the amount that is transferred from one location to another.	VARCHAR(10)	NOT NULL
rid	This is the uid of the receiver who is receiving the donation.	INT	NOT NULL
uid	This is the uid of the user who is donating to another user.	INT	FOREIGN KEY, NOT NULL

6. Table Name: Team

Table Description:

The table contains the users who are in a team and their data.

Fields	Description	Data Type	Constraints
tid	This is the unique team id which will be unique for every team	INT	PRIMARY KEY, NOT NULL
tname	This is the name of the team which user will provide.	VARCHAR(20)	NOT NULL

7. Team Name: User_Team

Team Description:

This table is created for user and team relationship so that we are able to fetch the matches.

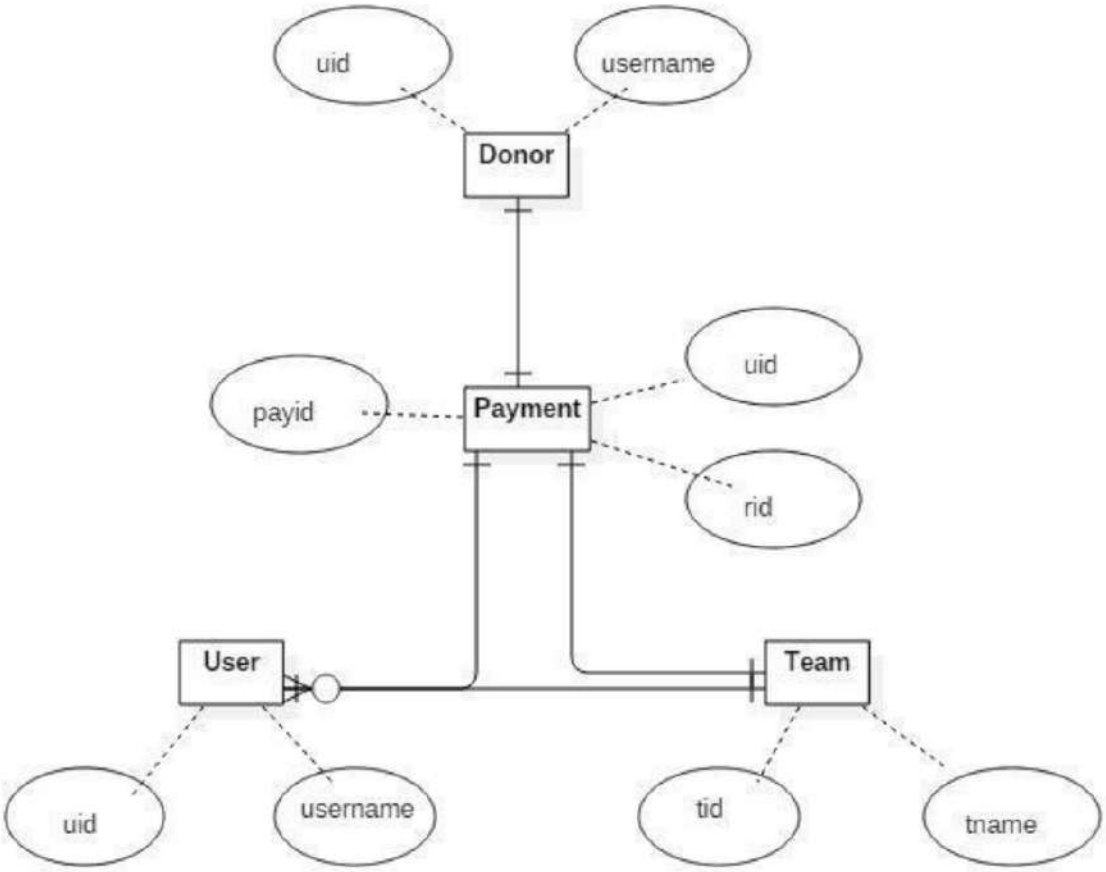
Fields	Description	Data Type	Constraints
uid	This is the userid of the user which is unique to every use.	INT	FOREIGN KEY
tid	This is the teamid of the team to which the user belongs to.	INT	FOREIGN KEY

4.4 EVENT TABLE

Sr. No	Event	Trigger	Source	Activity	Response	Destination
1.	Registration.	Register new user.	New User.	Add a new user.	Account created successfully.	Login Page.
2.	Logging in	Log in to the account	Existing User.	Authenticate and login the user his account.	Login successful.	Home Page.
3.	Profile Creation Settings	Edit profile	User	Add account details. Edit	Changes saved successfully.	Profile Page
4.		Edit settings	User	settings user adding a like team or setting splitting percentage.	Changes saved successfully.	Settings Page
5.	Team creation	Create team.	User	Creating a new team for more than one developers.	Team created successfully	Settings Page
6.	Adding users to the team	Add users.	User(Team Leader)	Adding the users of the to team using their username.	Users added Settings Page. the team successfully.	
7.	Removing users from the team.	Remove user.	User(Team Leader)	Removing a user from a team using their username.	User removed from the team successfully.	Settings Page.
8.	Adding splitting percentage for the funds.	Add splitting percentage.	User(Team Leader)	Adding splitting percentage of the funds for different members of the team.	Changes saved successfully.	Settings Page
9.	Linking GitHub account	Login with GitHub.	User	Linking GitHub account to the Contripay account for verification	Account Linked successfully.	Settings Page.

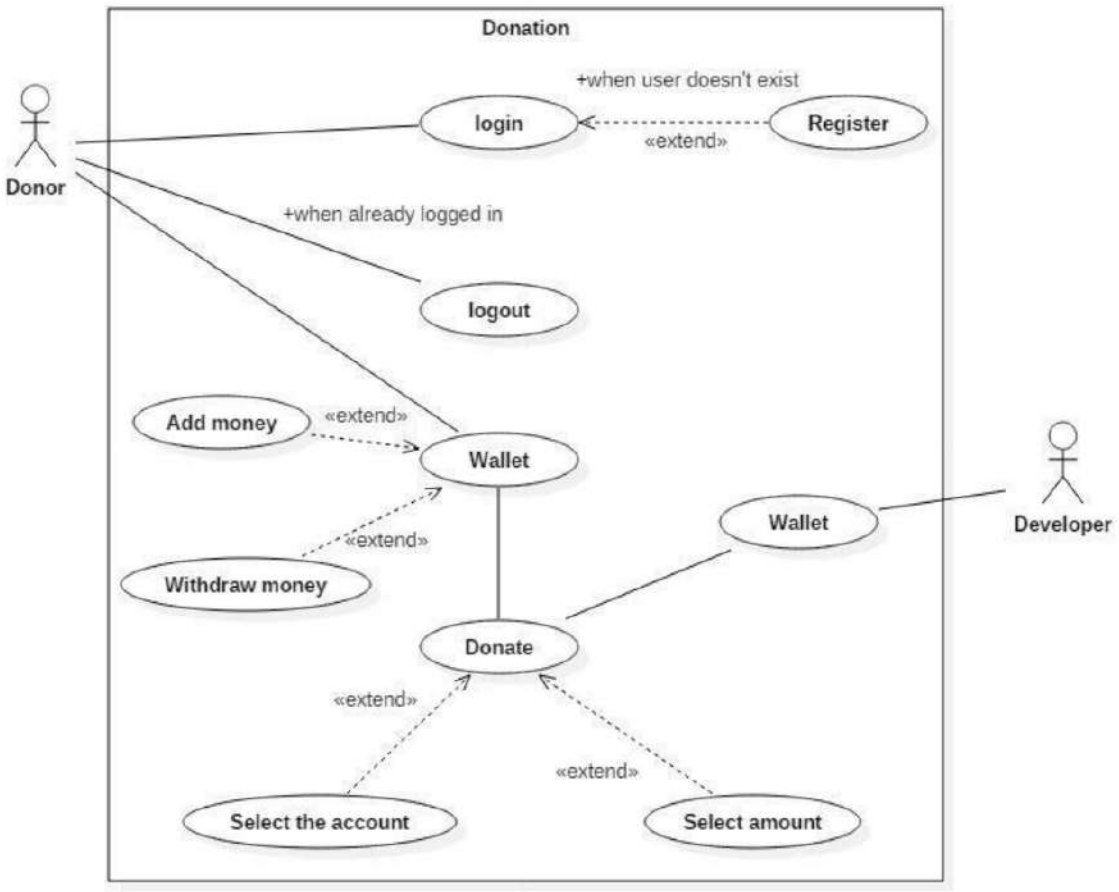
10.	Adding funds to the account	Add money	User	Adding money using cc/dc to the account.	Money added successfully.	Wallet Page.
11.	Withdrawing funds from the account.	Withdraw money.	User	Withdrawing money from Contripay account to the bank account.	Money withdrawn successfully.	Wallet Page.
12.	Creating a post.	Add a post.	User	Adding a post on the update wall for the donors.	Post added successfully	Update Wall

4.5 ER DIAGRAM

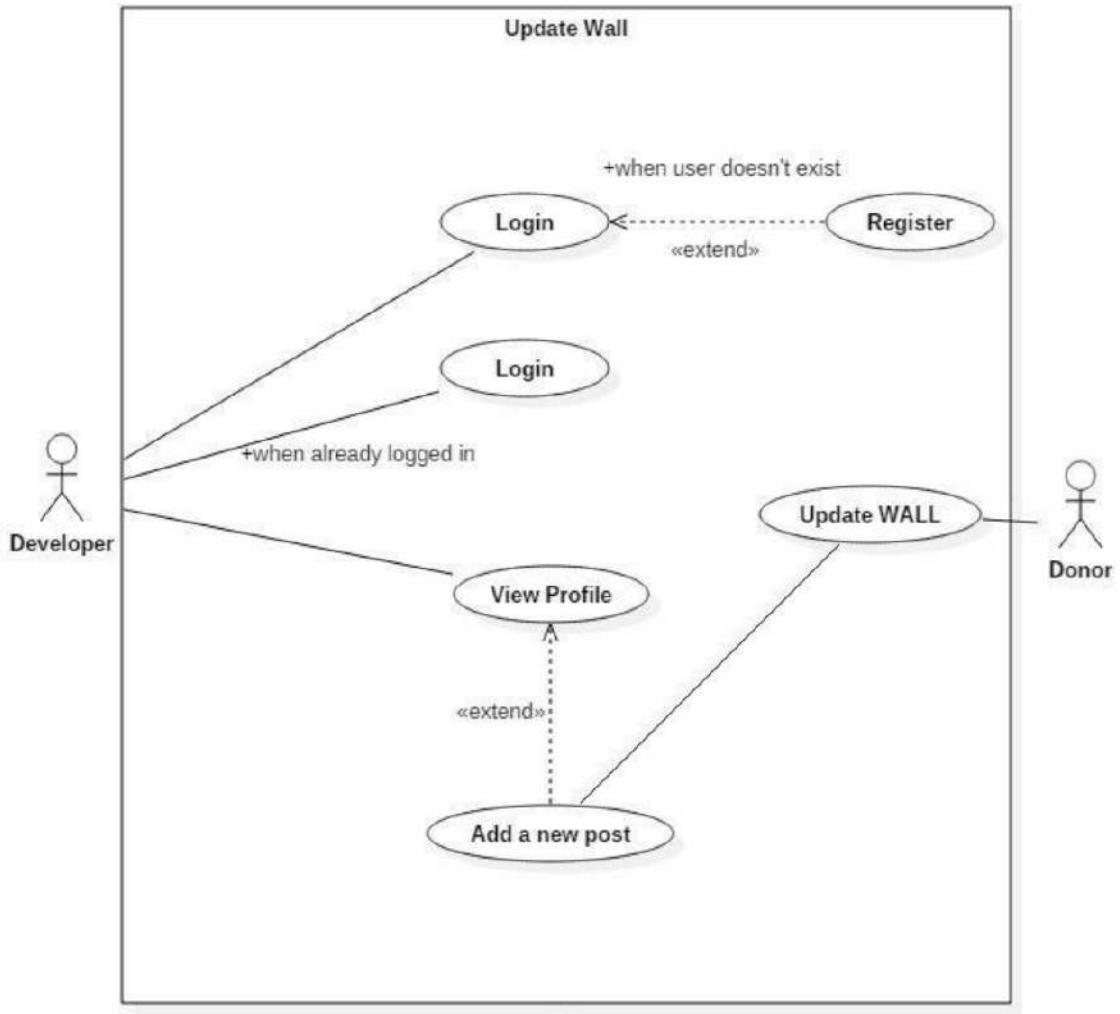


4.6 USE CASE DIAGRAM

DONATION

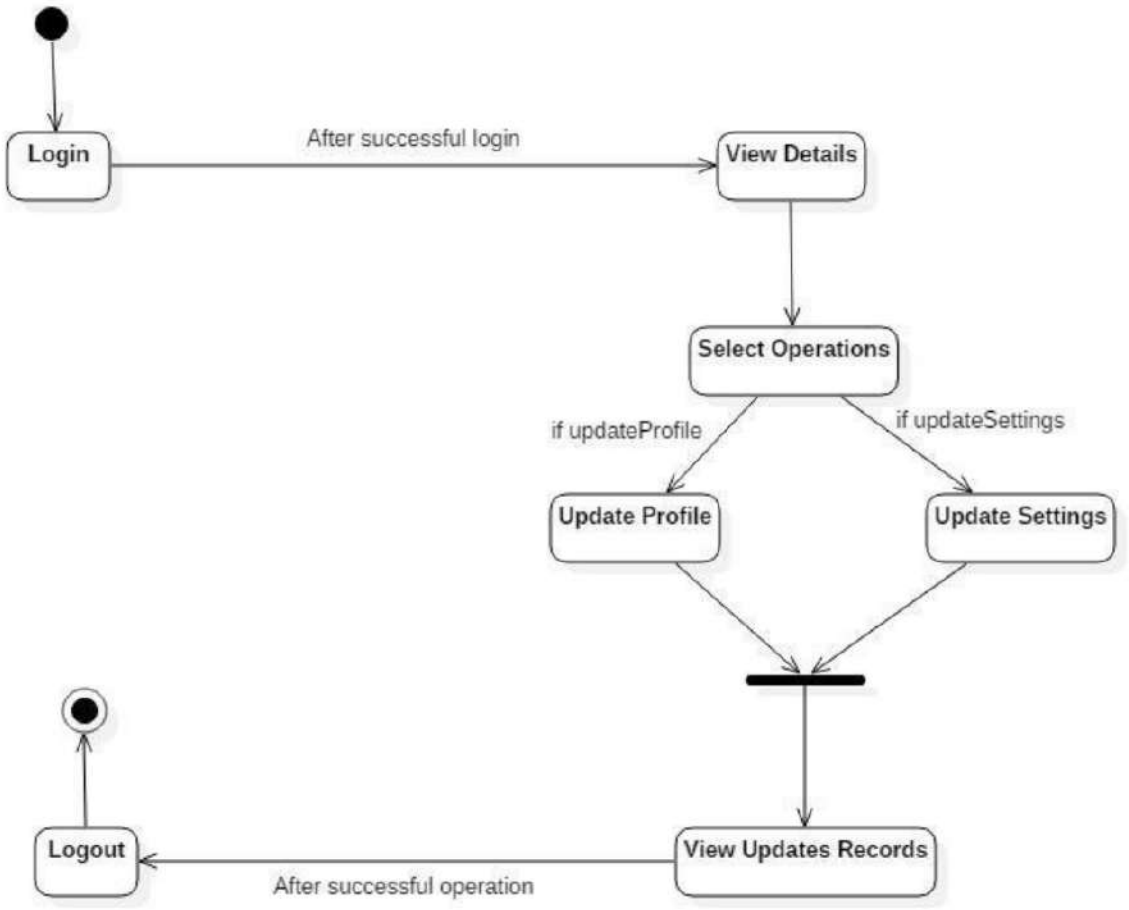


UPDATE WALL

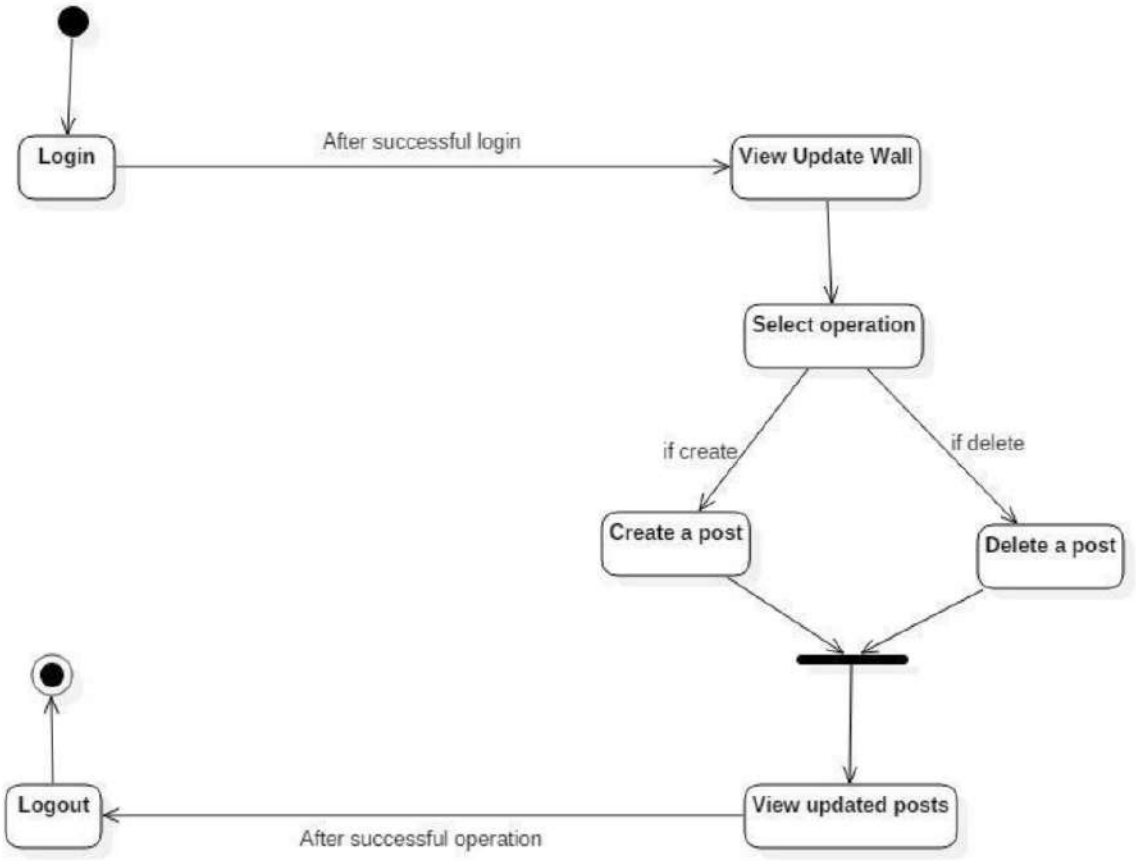

Update Wall
+when user doesn't exist
Login
Register
«extend»
Login
+when already logged in
Developer
Update WALL
Donor
View Profile
«extend»
Add a new post

4.7 STATE TRANSITION DIAGRAM

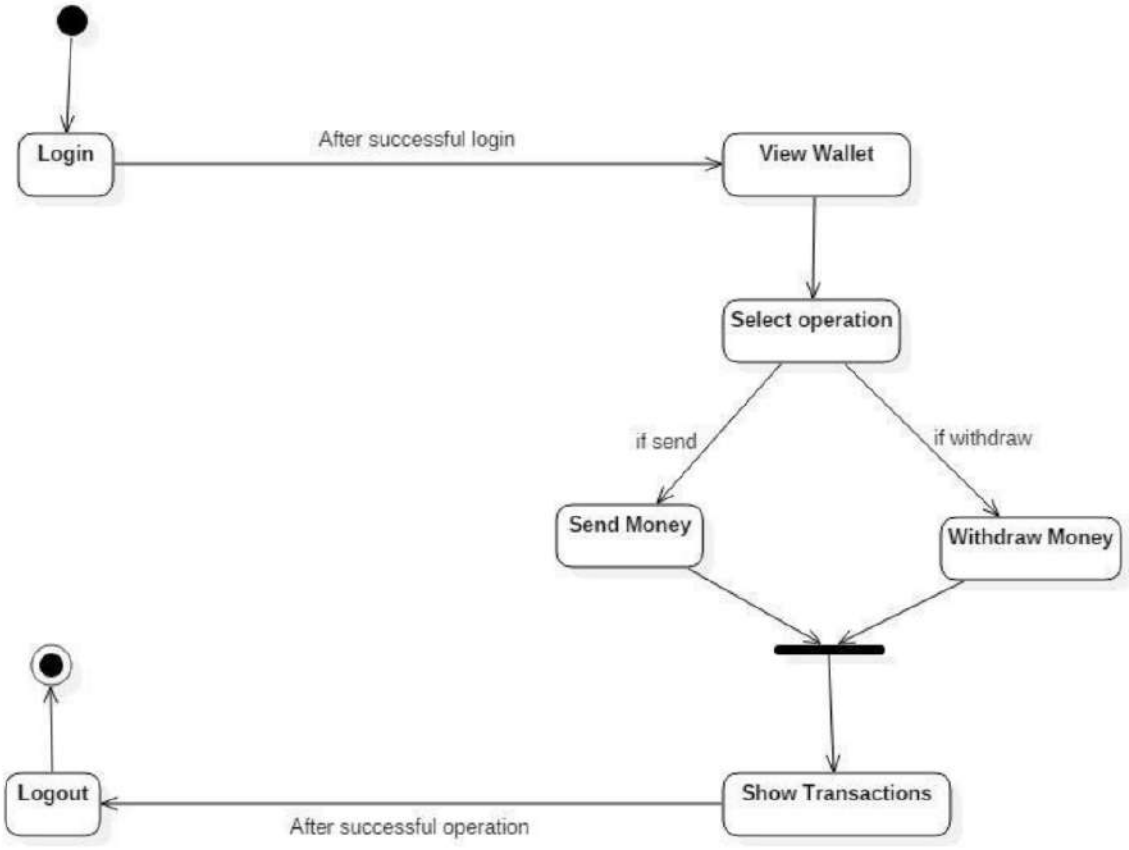
PROFILE/ACCOUNT



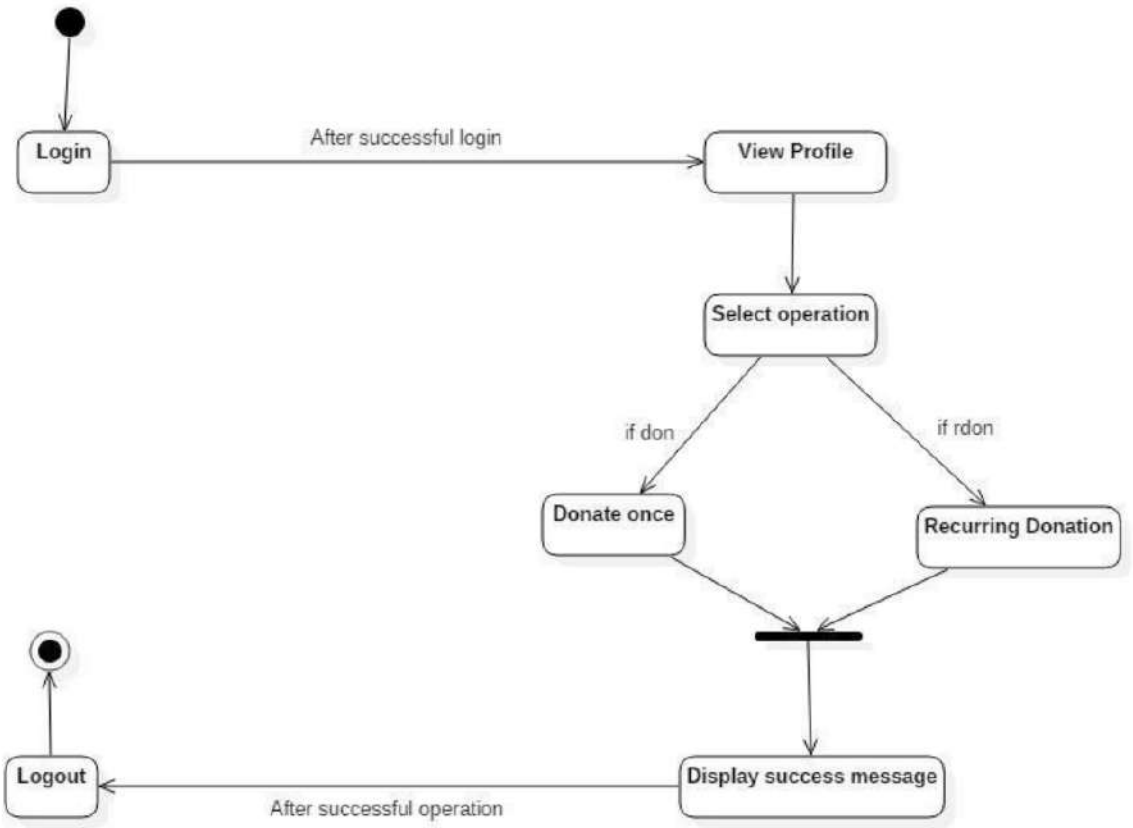
POSTS


Login
After successful login
View Update Wall
Select operation
if create
if delete
Create a post
Delete a post
Logout
After successful operation
View updated posts

WALLET

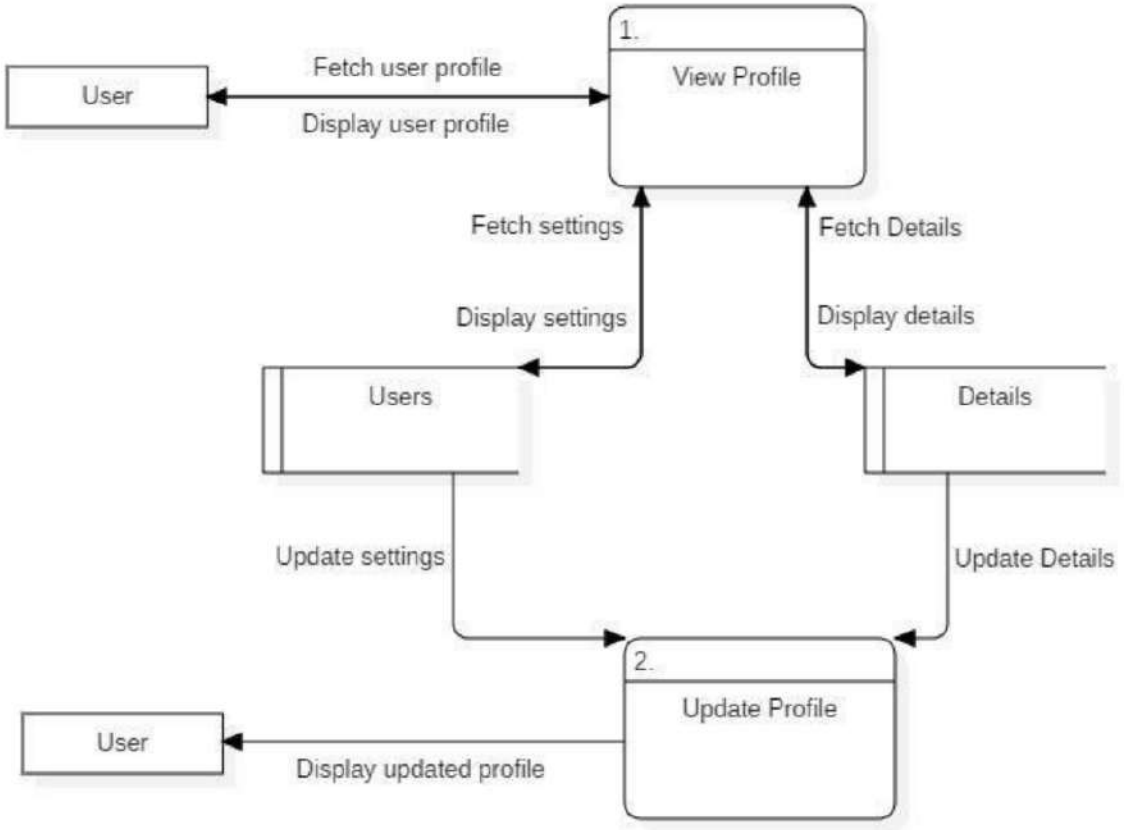

Login
After successful login
View Wallet
Select operation
if send
if withdraw
Send Money
Withdraw Money
Logout
Show Transactions
After successful operation

DONATION

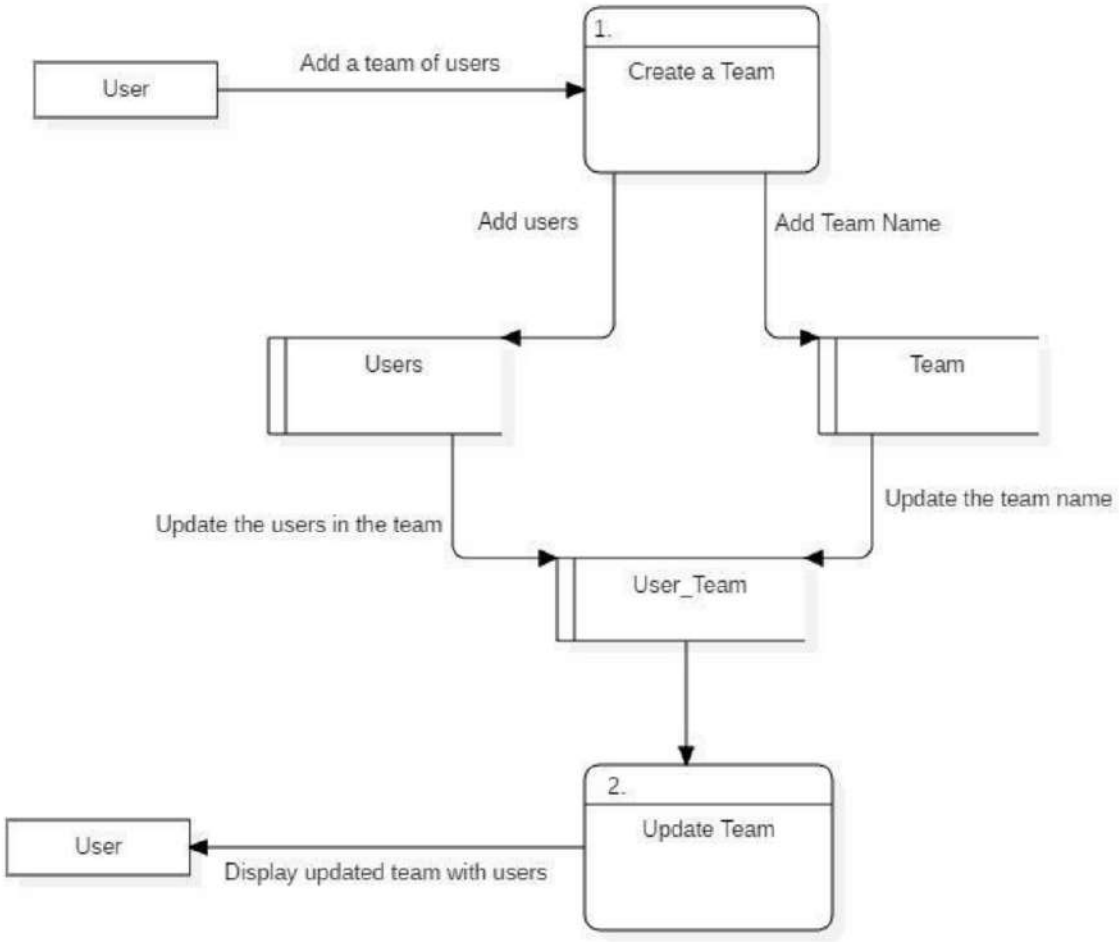

Login
After successful login
View Profile
Select operation
if don
if rdon
Donate once
Recurring Donation
Logout
After successful operation
Display success message

4.8 DATA FLOW DIAGRAMS

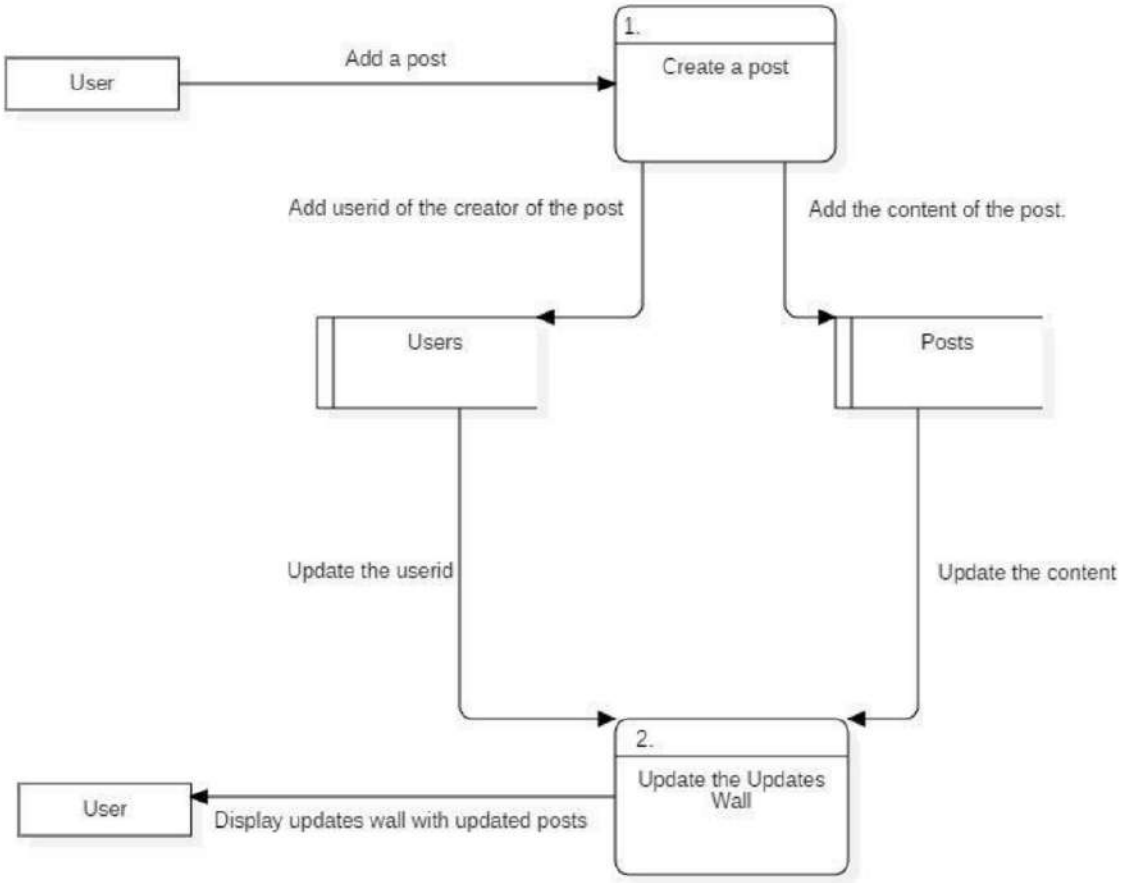
PROFILE/ACCOUNT



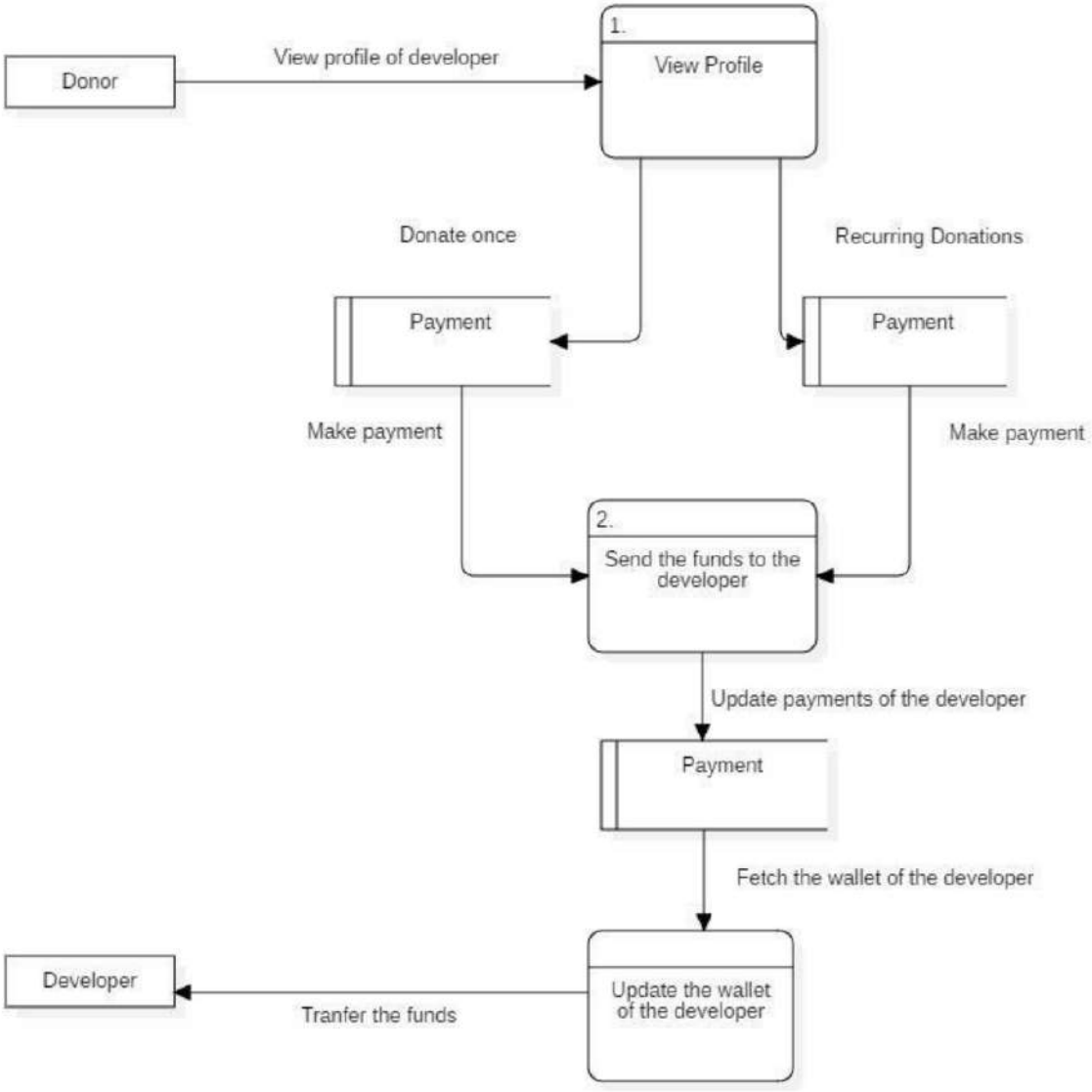
TEAM


User
Add a team of users
1.
Create a Team
Add users
Add Team Name
Users
Team
Update the users in the team
Update the team name
User_Team
2.
Update Team
User
Display updated team with users

UPDATE WALL

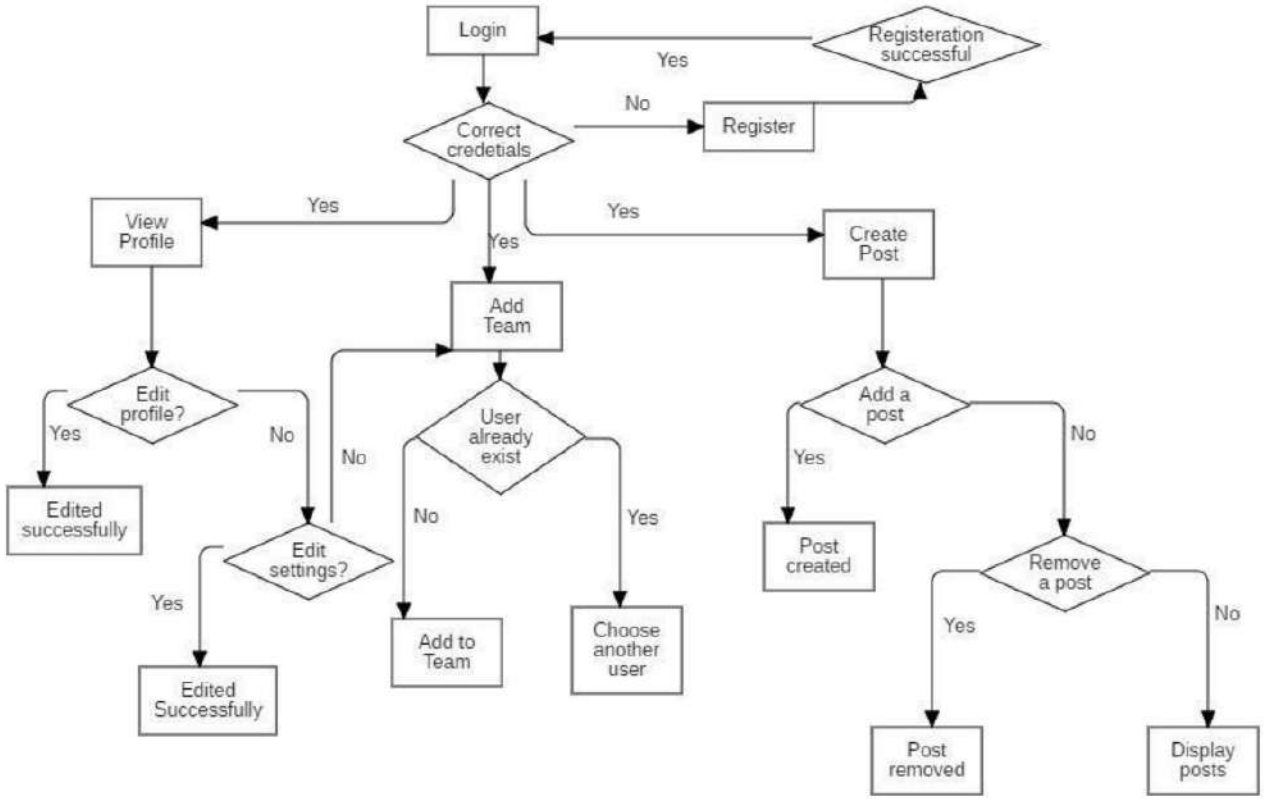

User
Add a post
1.
Create a post
Add userid of the creator of the post
Add the content of the post.
Users
Posts
Update the userid
Update the content
2.
Update the Updates Wall
User
Display updates wall with updated posts

DONATION/PAYMENT

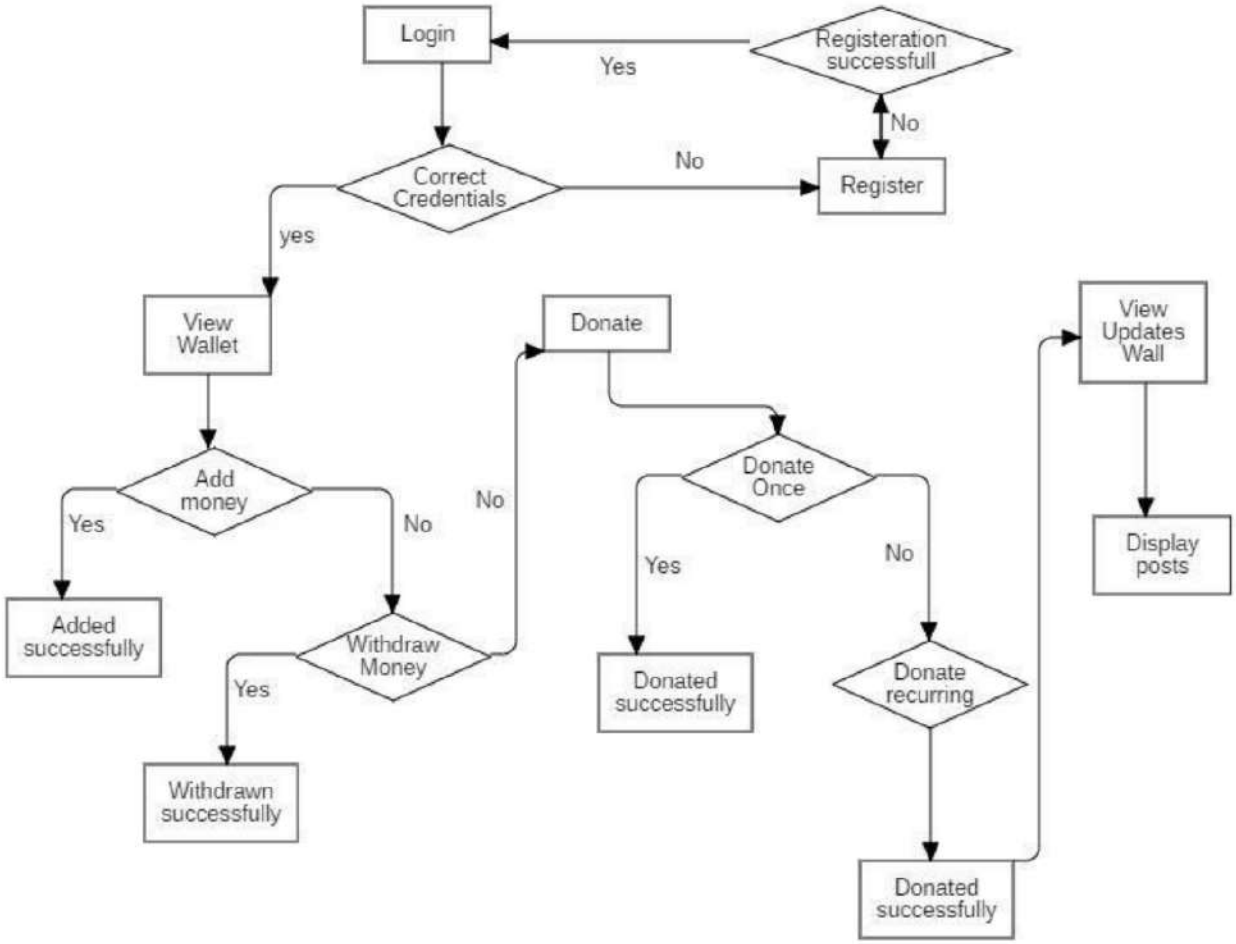


4.9 FLOW CHART

DEVELOPER



DONOR


Login
Registeration successfull
Yes
No
Correct Credentials
No
Register
yes
View Wallet
Donate
View Updates Wall
Add money
Yes
No
No
Donate Once
Display posts
Added successfully
Withdraw Money
Yes
No
Donated successfully
Donate recurring
Yes
Withdrawn successfully
Donated successfully

CHAPTER 5
IMPLEMENTATION AND TESTING

5.1.1. Important code and description

1. Hashing in register module:

```
app.post('/register', function (req, res)
{
    var username = req.body.username;
    var password = req.body.password;
    var email = req.body.email;
    var active = false;
    req.session.username = username;
    req.session.password = password;
    req.session.email = email;
    bcrypt.genSalt(10, function(err, salt)
    {
     bcrypt.hash(password, salt, function(err, hash)
     {
       pgpool.query('INSERT INTO "users" (username, password, active, email) VALUES ($1, $2, $3, $4)',
       [username, hash, active, email], function (err, result)
       {
        if (err)
        {
            res.status(500).send(err.toString());
        }
        else
        {
            res.redirect("/send");
        }
       });
     });
   });
});
```

Description:

Hashing is a technique used to convert readable confidential data into an unreadable encrypted format for privacy.

The above code is used to encrypt the password entered by the user during the registration. The bcrypt.genSalt function is used to generate a random salt. This salt is then used with the password to generate a hash using the bcrypt.hash function.

This will encrypt the user's password and this encrypted form of password is stored in the database. Hence even If an attacker gets the access to the database, he won't be able to access the password as hashed password can't be decrypted.

2. Comparing the hash in login module

Code:

```
pgpool.query('SELECT * FROM "users" WHERE username = $1', [username], function (err, result)
{
    if (result.rows.length !== 0)
    {
      req.session.uid= result.rows[0].id;
      var dbString = result.rows[0].password;
      var active= result.rows[0].active;
      bcrypt.compare(password, dbString, function(err, resu)
      {
        if (resu==true)
        {
        if(active==true)
        {
          req.session.username = username;
          req.session.password = password;
          console.log(result.rows[0].id);
          res.redirect('/dashboard');
        }
        else
        {
          req.session.destroy();
          res.render("login.hbs",{msg:"Please verify your account!"});
        }
        }
        else
        {
          res.render("login.hbs",{msg:"Invalid password!"});
        }
      });
```

As said above, the hashed passwords can't be decrypted or reversed by any means as the salt increases the entropy of the hash generated. So while logging in, we can't decrypt the password from the database and compare it with the password entered the user. For this purpose we use comparison.

The bcrypt.compare function is used to compare the hash of the password entered by the user with the hash stored in the database for that particular account. If the hash of the password entered by the user matches the hashed password in the database, then the user is authenticated.

3. Module for sending verification email.

Code:

```
app.get('/send',function(req,res){
  var text = req.session.username;
  var bytes = utf8.encode(text);
  var rand = base64.encode(bytes);

  link="https://contripay.herokuapp.com/verify?code="+rand;
  var email = {
    to: [req.session.email],
    from: 'admin@contripay.com',
    subject: 'Please confirm your Email account',
    text: 'Hey',
    html: "Hello,<br> Please Click on the link to verify your email.<br><a href="+link+">Click here to verify</a>"
  };

  mailer.sendMail(email, function(error, response) {
    if (error) {
      console.log(error) ;
      res.send(error);
    }
    console.log(response);
    res.render("register.hbs",{msg:"Email sent successfully! Please verify your account"});
  });

});
```

Description:

Node js comes with an npm (Node Package Manager) which is used to install third party packages which contains libraries and functions helpful in building some functionalities. Node mailer is one of those packages which helps to send emails using node js. Node Mailer uses SMTP protocol to send emails. When the user register for a new account, this endpoint is called which send the verification mail to the users so that the identity of the user can be authenticated.

Sendgrid is an email delivery service which delivers your transactional and marketing emails through the world's largest cloud-based email delivery platform. In this module Sendgrid API is used for sending the verification emails.

5.1.2. Coding Efficiency

Node.js is a JavaScript runtime environment built on Chrome's V8 JavaScript engine. Node really provides is a very practical and optimized solution for web development. Node is quite fast and V8 engine is much optimized to perform simple CPU bound operations.

One of the great features is that you can create a REST API in node js which helps in accessing resources with less efficiency and more speed. In the proposed system there are many endpoints like /tid which is used at many places inside other endpoints to get the resource faster without querying the database or writing the query again and again. Also the node packages makes the code less complicated and lesser in number too as inbuilt libraries can be called directly.

5.2 Testing approaches

5.2.1. Unit Testing

Unit testing, a testing technique using which individual modules are tested to determine if there are any issues by the developer himself. It is concerned with functional correctness of the standalone modules.

The main aim is to isolate each unit of the system to identify, analyse and fix the defects.

Advantages:

Reduces Defects in the newly developed features or reduces bugs when changing the existing functionality.

Reduces Cost of Testing as defects are captured in very early phase.

- Improves design and allows better refactoring of code.

Unit Tests, when integrated with build gives the quality of the build as well.

In the proposed system the unit testing is done by testing each module separately using test cases.

Every module has a test case which includes different steps to perform and test a module so as to check its functioning.

The expected and the actual result is matched and the test is declared as passed or failed on the basis of that.

Following are the test cases for the proposed system under Unit Testing:

Project Name: Contripay	
Test Case	
Test Case ID: 1	Test Designed by: Adarsh
Test Priority (Low/Medium/High): High	Test Designed date: 25-01-2019
Module Name: Login	Test Executed by: Adarsh
Test Title: Login	Test Execution date: 25-01-2019
Description: Used to login user into his/her account.	
Pre-conditions: User should be a registered user using the register module	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter username & password in respective text fields	Correct username & incorrect password	Password is incorrect	Password is incorrect	Pass	
2	Enter username & password in respective text fields	Incorrect username & correct password	User does not exist	User does not exist	Pass	
3	Enter username & password in respective text fields	Incorrect username & incorrect password	Username/ Password is incorrect	Username/ Password is incorrect	Pass	
4	Enter username & password in respective text fields	Correct username & correct password	Logged in successfully	Logged in successfully	Pass	

Post-conditions:

User is logged in successfully If the correct credentials are given & should be again redirected to login page with an error prompt If the credentials are incorrect

Test Case	
Test Case ID: 2	Test Designed by: Adarsh
Test Priority (Low/Medium/High): High	Test Designed date: 25-01-2019
Module Name: Register	Test Executed by: Adarsh
Test Title: Register	Test Execution date: 25-01-2019
Description: Used to register a new account for the user.	
Pre-conditions:	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter username & password in respective text fields	Correct username & incorrect password	Password is incorrect	Password is incorrect	Pass	
2	Enter username & password in respective text fields	Incorrect username & correct password	User does not exist	User does not exist	Pass	
3	Enter username & password in respective text fields	Incorrect username & incorrect password	Username/Password is incorrect	Username/Password is incorrect	Pass	
4	Enter username & password in respective text fields	Correct username & correct password	Logged in successfully	Logged in successfully	Pass	

Post-conditions:

User is logged in successfully If the correct credentials are given & should be again redirected to login page with an error prompt If the credentials are incorrect

Project Name: Contripay	
Test Case	
Test Case ID: 3	Test Designed by: Adarsh
Test Priority (Low/Medium/High): Low	Test Designed date: 25-01-2019
Module Name: Account Settings	Test Executed by: Adarsh
Test Title: Settings	Test Execution date: 25-01-2019
Description: Used to change settings of an account	
Pre-conditions: User should be a registered user and should be logged in to the account.	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter same username & different password	Same username & different password	Password updated successfully	Password updated successfully	Pass	
2	Enter different username & different password	Incorrect username & correct password	Username & password updated successfully	Username & password updated successfully	Pass	
3	Enter different existing username & same password in respective text fields	Incorrect username & incorrect password	Username already exists	Username already exists	Pass	
4	Enter different non-existing username & same password in respective text fields	Correct username & correct password	Username & password updated successfully	Username & password updated successfully	Pass	

Post-conditions:
User is logged in successfully If the correct credentials are given & should be again redirected to login page with an error prompt If the credentials are incorrect

Project Name: Contripay	
Test Case	
Test Case ID: 4	Test Designed by: Adarsh
Test Priority (Low/Medium/High): High	Test Designed date: 25-01-2019
Module Name: Create Team	Test Executed by: Adarsh
Test Title: Create Team	Test Execution date: 25-01-2019
Description: Used to create a team	
Pre-conditions: User should be a registered user and should be logged in to the account.	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter an existing team name	Existing team name	Team name already exists	Team name already exists	Pass	
2	Enter a non-existing team name	Non-existing team name	Team created successfully	Team created successfully	Pass	

Post-conditions:

User is logged in successfully If the correct credentials are given & should be again redirected to login page with an error prompt If the credentials are incorrect

Project Name: Contripay	
Test Case	
Test Case ID: 5	Test Designed by: Adarsh
Test Priority (Low/Medium/High): Medium	Test Designed date: 25-01-2019
Module Name: Team Settings	Test Executed by: Adarsh
Test Title: Team Settings	Test Execution date: 25-01-2019
Description: Used to edit Team details	
Pre-conditions: User should be a registered user and should be logged in to the account and should be admin of the team	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter Team name	Existing Team name	Team name already exists	Team name already exists	Pass	
2	Enter Team name	Non-existing Team name	Team details updated successfully	Team details updated successfully	Pass	
3	Enter Team name and description	Existing Team name and description	Team name already exists	Team name already exists	Pass	
4	Enter Team Name and description	Non-existing Team name and description	Team details updated successfully	Team details updated successfully	Pass	

Post-conditions:

The team details will be updated If a non-existing team name is used for updating of Team details

Test Case	
Test Case ID: 6	Test Designed by: Adarsh
Test Priority (Low/Medium/High): Medium	Test Designed date: 25-01-2019
Module Name: Add user	Test Executed by: Adarsh
Test Title: Add user	Test Execution date: 25-01-2019
Description: Used to add a member to the team	
Pre-conditions: User should be a registered user and should be logged in to the account and should be admin of the team	
Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter username	Username which does not exist	User does not exist	User does not exist	Pass	
2	Enter username	Username which exists but is not verified	User is not verified	User is not verified	Pass	
3	Enter username	Username which exists and is verified	User added to the team successfully	User added to the team successfully	Pass	

Post-conditions:

The user specified will be added to the team successfully and will be given least privileges

Test Case ID: 7	Test Designed by: Adarsh
Test Priority (Low/Medium/High): Medium Module Name: Post	Test Designed date: 25-01-2019 Test Executed by: Adarsh
Test Title: Post	Test Execution date: 25-01-2019
Description: Used to post messages or updates on the wall	
Pre-conditions: User should be a registered user and should be logged in to the account or should be admin of the team Dependencies:	

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Enter text for the post	Post message	Post updated successfully	Post updated successfully	Pass	
2	Delete the post	Admin	Post deleted successfully	Post deleted successfully	Pass	
3	Delete the post	Not admin	Not allowed to delete the post	Not allowed to delete the post	Pass	
4	Enter text for the post	Non-existing Team name and description	Team details updated successfully	Team details updated successfully	Pass	

Post-conditions:

The team details will be updated If a non-existing team name is used for updating of Team details

5.2.2. Integration Testing

Upon completion of unit testing, the units or modules are to be integrated which gives raise to integration testing. The purpose of integration testing is to verify the functional, performance, and reliability between the modules that are integrated.

Test Case ID	Test Case Objective	Test Case Description	Expected Result	Actual Result	Test Case Pass/Fail
1	Check the interface link between the Login and the Dashboard Check the interface between the registration page and login page	Enter login credentials and click on the Login button	To be directed to the Dashboard	Redirected to the dashboard page Redirected to the	Pass
2		Enter registration details and click on register button	To be redirected to the Login	login page	Pass
3	Check the interface link between the dashboard and the account settings	From Dashboard select the account settings option	To be redirected to account settings page	Redirected to account settings page	Pass
4	Check the interface link between the dashboard & create team page	From dashboard click on create a team	To be redirected to the create team page	Redirected to create team page	Pass
5	Check the interface link between dashboard and the team page	From dashboard click on team	List of the teams of which user is a part of	Redirected to list of the teams of which user is a part of	Pass
6	Check the interface team page and edit team page	From team page click on edit in front of the team you want to edit	To be redirected to the edit team page of the selected team	Redirected to the edit team page of the selected team	Pass

CHAPTER 6

RESULTS AND DISCUSSION

6.1 Test Reports

The test results are promising as all the test cases are passed and the proposed system can go through all the conditions set while testing it. It can face problematic situations and can work fine in different & difficult conditions.

6.1.1 Project Information

<table>
<tr><td colspan="5">Project Overview</td></tr>
<tr><td colspan="5">PROJECT BASIC INFORMATION</td></tr>
<tr><td>Project Name</td><td colspan="4">Contripay</td></tr>
<tr><td>Product Description</td><td colspan="4">A crowd funding platform.</td></tr>
<tr><td rowspan="2">Project Description</td><td colspan="4">Contripay is a crowd funding platform for open source developers where the open source developers can showcase their work & get financial support from the people who appreciate their work.</td></tr>
<tr><td colspan="2">Project Type</td><td colspan="2">Web Application</td></tr>
<tr><td>Project Duration</td><td>Start date</td><td>27th Aug 2018</td><td>End date</td><td>30th April 2019</td></tr>
</table>

6.1.2 Test Objectives

The testing of the proposed system had different goals and objectives. The major objectives of testing are as follows:

- Finding defects & bugs which may be introduced by the programmer while developing the proposed system.
- Gaining confidence in the product and providing information about the level of quality to the users of the application.
- To prevent & avoid any defects and failures.
- To make sure that the end result meets the business and user requirements.
- To ensure that it satisfies the BRS that is Business Requirement Specification and SRS that is System Requirement Specifications as both are necessary as one satisfies the customers & another satisfies the developer.
- To gain the confidence of the customers by providing them a quality product and make them have trust in the product.

6.1.3 Test Summary

Test Cycle System Test

EXECUTED	PASSED				7
	FAILED				0
	(Total) TEST EXECUTED (PASSED + FAILED)				7
PENDING					0
IN PROGRESS					0
BLOCKED					0
(Sub- Total) TEST PLANNED (PENDING + IN PROGRESS + BLOCKED + TEST EXECUTES)					7

Functions	Description	% TC's Executed	% TC's Passed	TCs pending	Priority	Remarks
Login	Used to login user his/her account. Used to register a	100%	100%	0	High	
Register	new account for the user Used to change	100%	100%	0	High	
Account settings	settings of an account Used to create	100%	100%	0	Low	
Create Team	a team	100%	100%	0	High	
Team Settings	Used to edit Team details	100%	100%	0	Medium	
Add User	Used to add a member to the team	100%	100%	0	Medium	
Posts	Used to post messages or updates on the wall	100%	100%	0	Medium	

6.1.4 Defects

Total No. of bugs: 0

Status of bugs: Closed

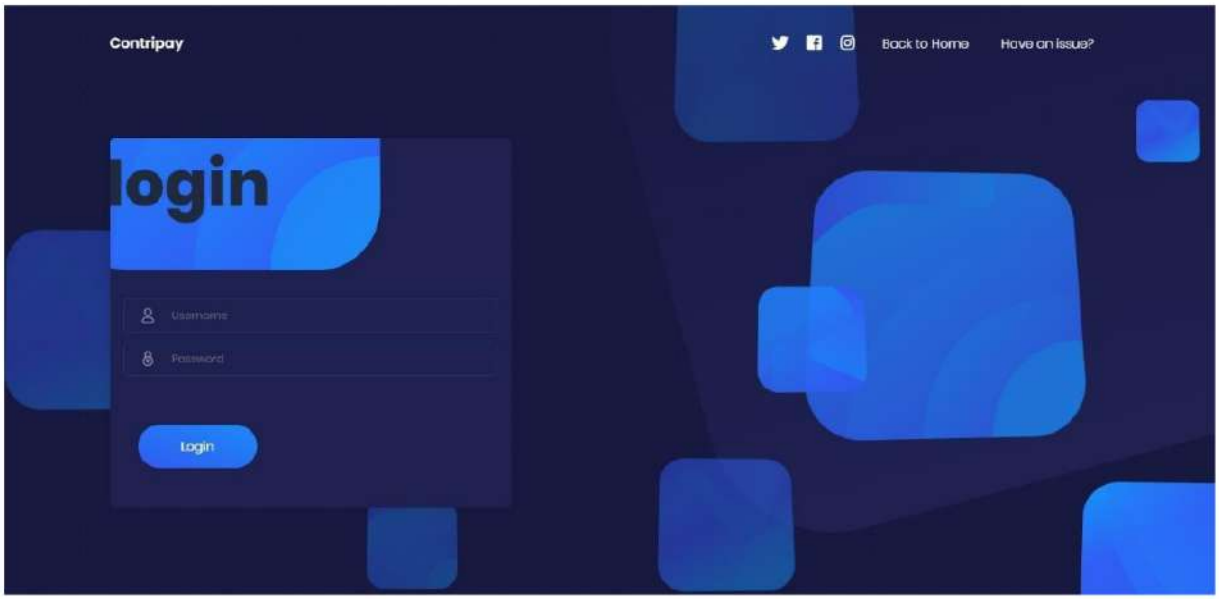
6.2 User Documentation

1. Home



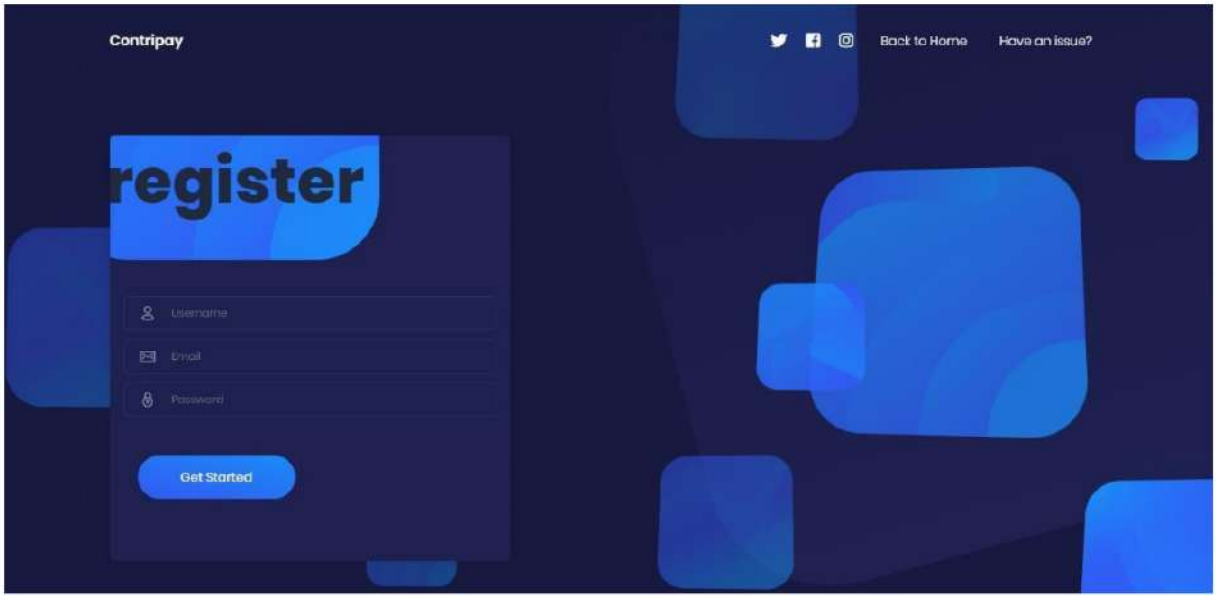
Gives a descriptive information to the users who are visiting the website for the first time. The user gets to know what the web application is about and helps them to get started.

2. Login



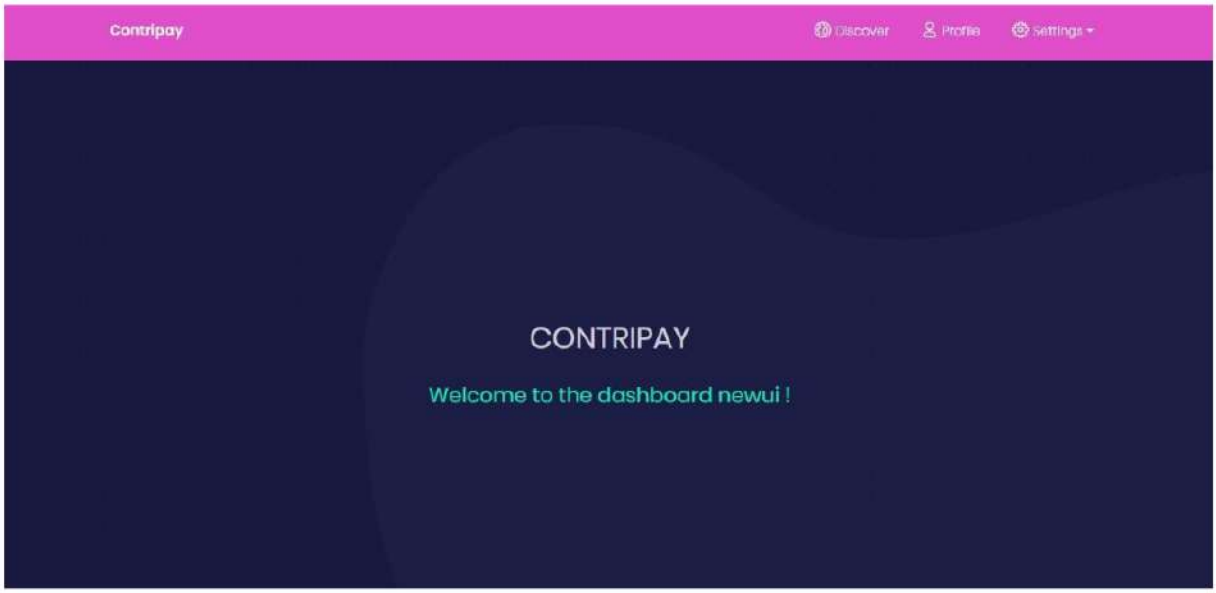
This is the interface from where the registered user can log into their account using their username& password.

3. Register



This is the interface through which an unregistered user can create an account using his/her personal details like email, a password for security & a username for identitiy.

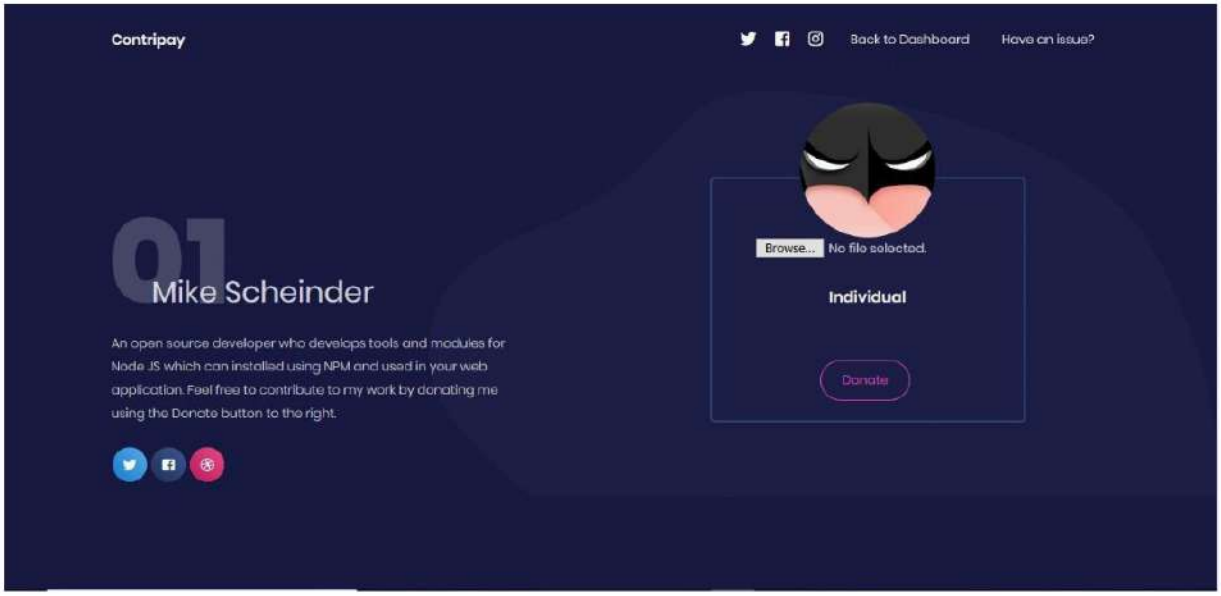
4. Dashboard



This is the first page that the user encounters when logged into his account & lets the user know that he/she is logged in.

Also, we have different options to work with in the menu.

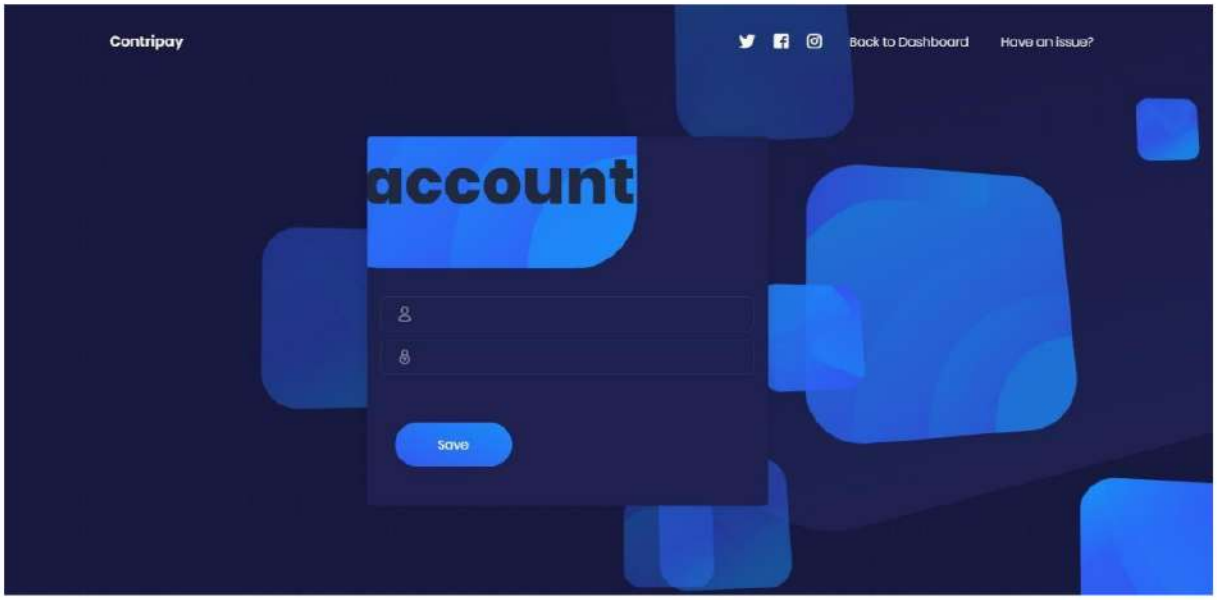
5. Profile



This is a profile page of a user on Contripay which will be used to get funding from the supporters by sharing your profile page with them.

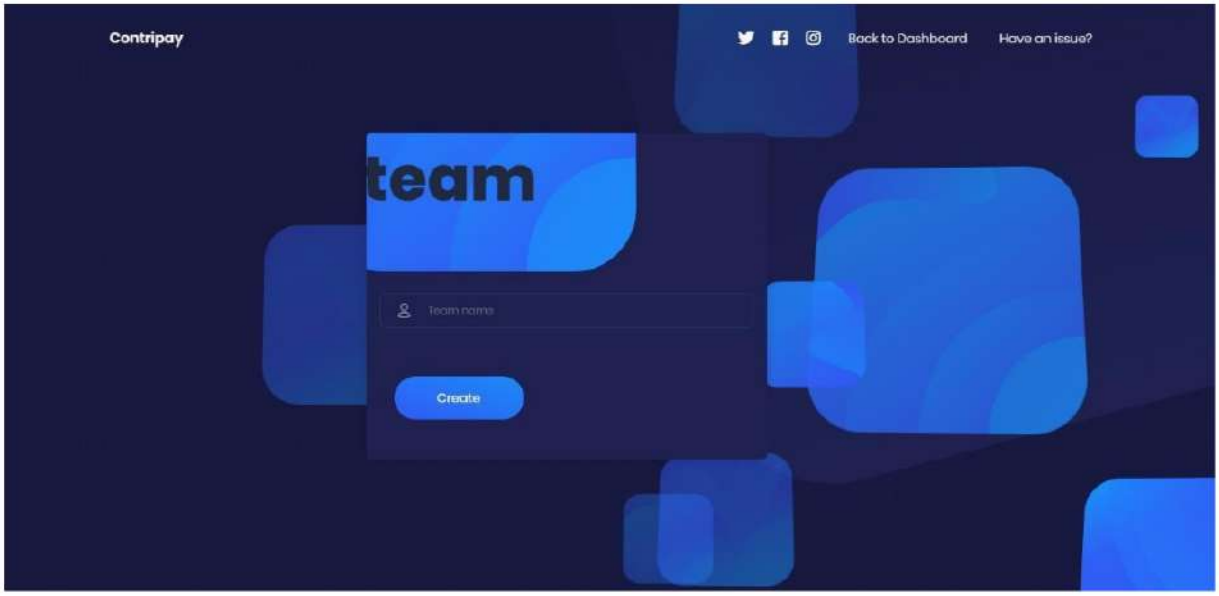
Also, it gives the basic information about the person or the team of whom the profile is.

6. Account Settings



We can change our account settings & details like our username or password using this page.

7. Create Team



We can create a team of many members using this interface by giving a name for the team to be created.

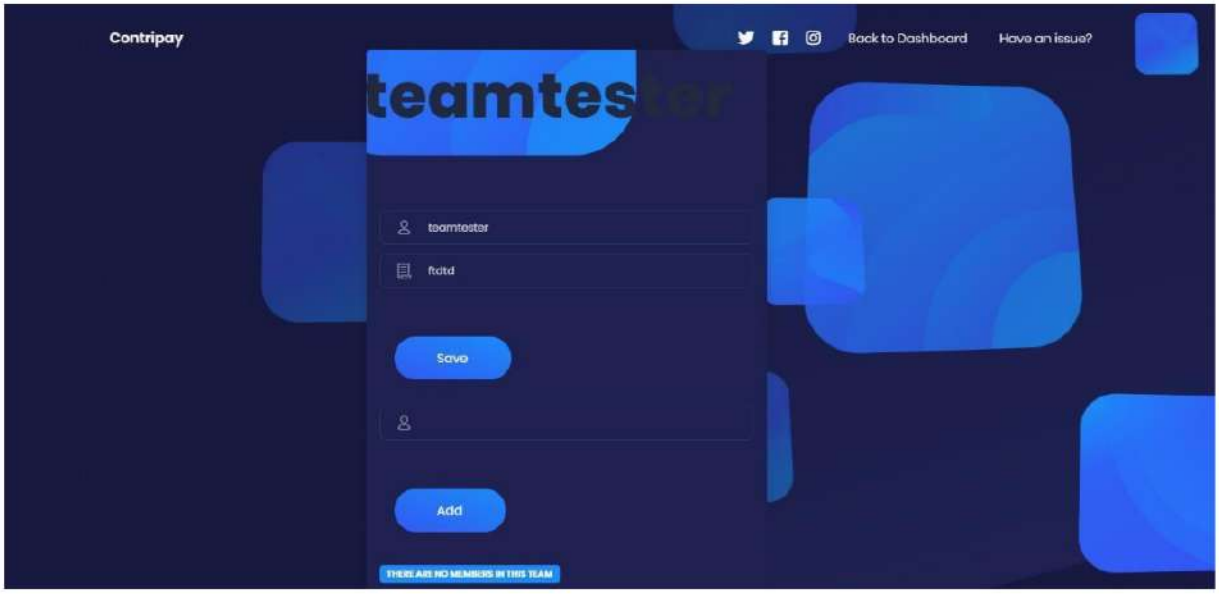
8. Team List



This interface will list and show all the team that you are a part of.

You can select any team of which you want to change settings or details of.

9. Edit Team



This interface will let u edit the setting of the team that you selected from the previous interface.

It will let you change the name & description of the team which you've created.

Also, it lets you add & remove new members to your team.

CHAPTER 7

CONCLUSION

7.1 Conclusion "Contripay" is developed to support & help the open source developers financially to help
them maintain their work. Using the proposed system developers can collect funds from different people or the users using their project/product. It also serves as a platform for the open
source developers.

The main aim of the project is to provide financial support to the open source developers &
help them maintain their projects.

The proposed system:

1. The developers should be able to create an account with their GitHub account linked to it for receiving the donations.
2. The developers should also be able to create teams if there are more than one person working on a project.
3. The donors will get the new updates and the beta updates of the software before anyone else exclusively by the software developers as a gesture of thanking them.
4. The updates will be given through posts which can be posted by the members of the team or an individual.

Thus "Contripay" is an efficient & effective web application to fulfil the needs of the open source developers or anyone who want to get support from the people for a good cause.

7.2 Limitations of the system

There is some limitation in every web application which may be due to different constraints that don't support the system. There are also some limitations in the proposed system and they are as follows:

1. The user is not able to store & collect the funds as a wallet is not being developed to store the funds & withdraw it whenever they want.
2. All the donations are instant & can't be scheduled for later.
3. The system is hosted on a shared hosting and hence can't take load of large number of users but can handle it when it is scaled further.

7.3 Future Scope

The future scope of the application is wide as there is always space for improvement & the application is scalable to higher levels.

1. A separate wallet will be introduced to store & withdraw funds to & from an account.
2. The fund will be distributed within the members of the team.
3. The developer who creates and manages the team should be able to set the percentage of the contribution which will be received by the other members.
4. So, whenever someone donates them, the money is distributed between them according to that percentage.
5. The donors would also be able to set repetitive donations more like a monthly subscription to a project with the amount of their choice.

7.4References

https://www.patreon.com/
https://www.kickstarter.com/
https://www.indiegogo.com/
https://stackoverflow.com/
https://www.w3schools.com/
https://www.tutorialspoint.com/
https://nodejs.org/en/
https://www.postgresql.org/
https://www.heroku.com
https://www.ketto.org/